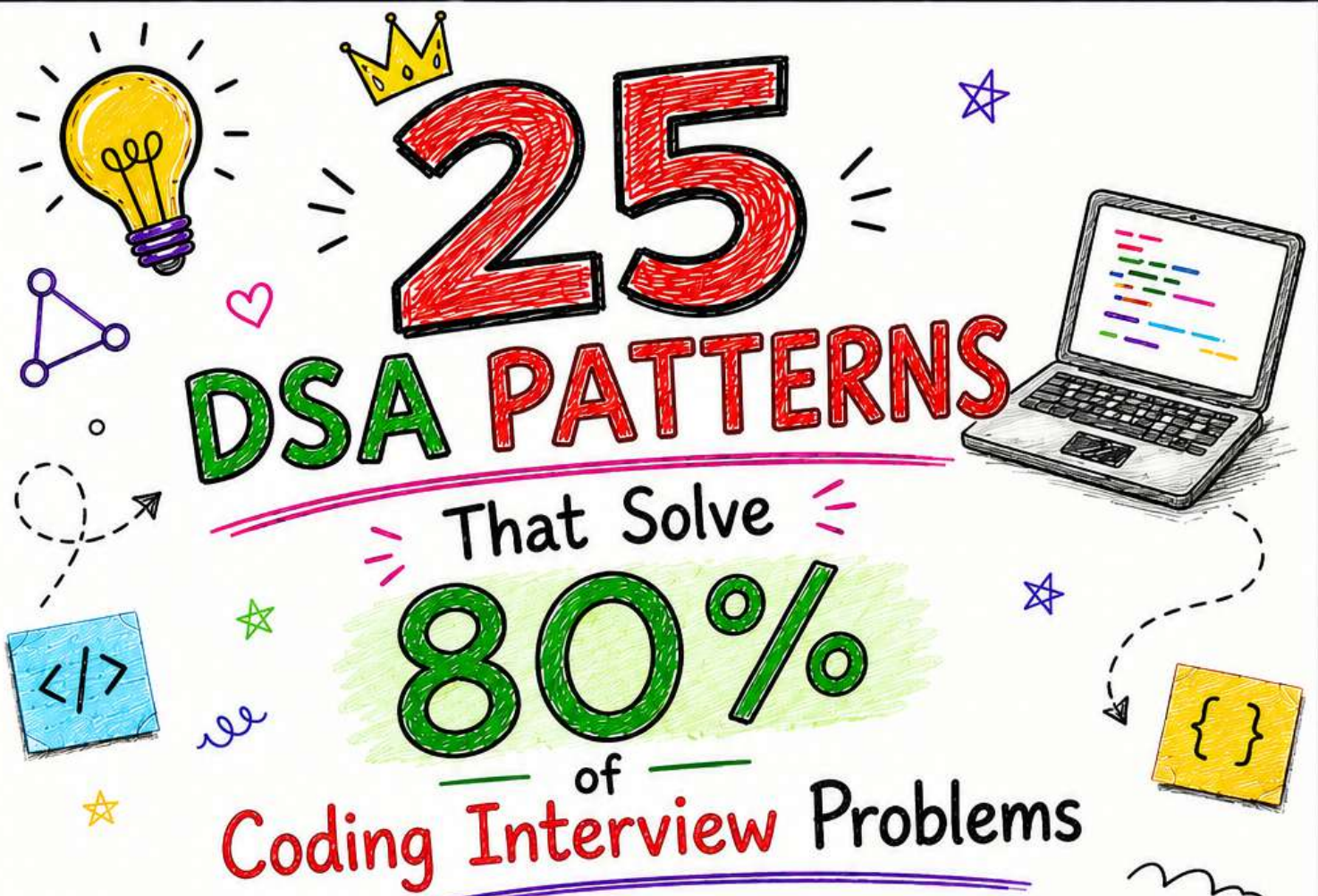


25 DSA PATTERNS

That Solve 80% of Coding Interview Problems



1 SLIDING WINDOW

Used to solve problems on subarrays / substrings.

Example: Longest Substring Without Repeating Characters

2 TWO POINTERS

Efficient technique for arrays / strings.

Example: 3Sum, Container With Most Water

3 FAST & SLOW POINTERS

Useful for linked list cycle detection.

Example: Linked List Cycle

4 MERGE INTERVALS

Merge overlapping intervals.

Example: Merge Overlapping Intervals

5 BINARY SEARCH

Search in sorted arrays or answer space.

Example: Koko Eating Bananas

6 BIT MANIPULATION

Solve problems using bit operations.

Example: Single Number

7 PREFIX SUM

Precompute prefix sums for range queries.

Example: Range Sum Query

8 BACKTRACKING

Solve by trying all possible combinations.

Example: N-Queens Problem

9 DFS (DEPTH FIRST SEARCH)

Explore as far as possible in each direction.

Example: Number of Islands

10 BFS (BREADTH FIRST SEARCH)

Level by level exploration.

Example: Shortest Path in Unweighted Graph

11 HEAP / PRIORITY QUEUE

Quick access to min / max element.

Example: Top K Frequent Elements

12 RECURSION & DIVIDE & CONQUER

Break problem into smaller parts.

Example: Merge Sort

13 DYNAMIC PROGRAMMING

Store results of subproblems.

Example: Longest Increasing Subsequence

14 GREEDY

Make optimal choice at each step.

Example: Activity Selection Problem

15 GRAPH PATTERNS

Used for graph related problems.

Example: Detect Cycle in Graph

16 TRIE (PREFIX TREE)

Efficient for string related problems.

Example: Implement Trie

TIPS

- ✓ Identify the pattern first.
- ✓ Practice similar problems.
- ✓ Understand the approach deeply.
- ✓ Dry run with examples.
- ✓ Code with clean & optimal solution.

Do follow @AdarsChetan on LinkedIn & X for more such amazing stuff!

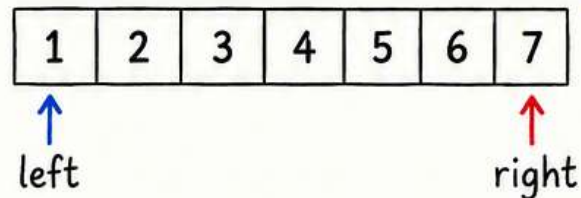
- @adarschetan

ARRAY & POINTER PATTERNS

1 Two Pointers

Use two indices moving through an array or string to reduce nested loops.

Best For: Pair sum, palindrome, removing duplicates.

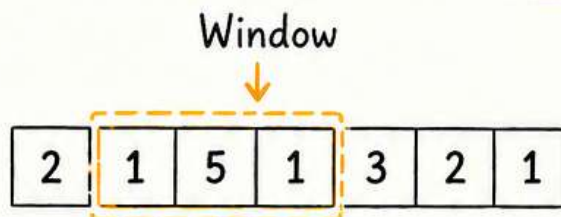


Move both pointers towards center or in opposite direction.

2 Sliding Window

Maintain a dynamic range of elements instead of recalculating every subarray.

Best For: Longest substring, maximum sum subarray.

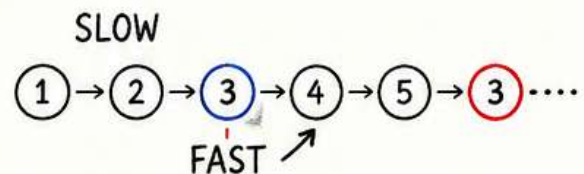


Expand or shrink the window based on the condition to get the optimal result.

3 Fast & Slow Pointers

Move one pointer faster than another to detect cycles or find middle elements.

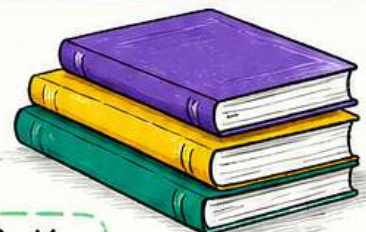
Best For: Linked lists, cycle detection.



Fast pointer moves 2 steps, slow pointer moves 1 step. If they meet, there is a cycle.

Note:

These patterns help solve problems efficiently by optimizing time complexity and reducing unnecessary calculations.



Do follow @AdarsChetan on LinkedIn & X for more such amazing stuff!



- @adarschetan

@adarschetan
Do follow me on
LinkedIn & X for
more such
amazing stuff! ☆

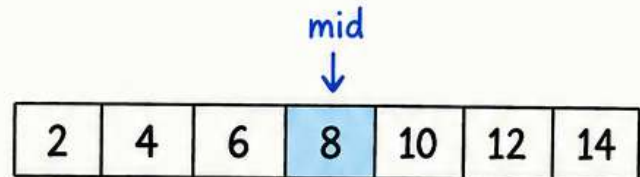
SEARCHING PATTERNS



4 Binary Search

Repeatedly divide the search space into halves.

Best For: Sorted arrays and efficient searching.



Ignore right half if target < mid

Ignore left half if target > mid

5 Modified Binary Search

Adapt binary search for rotated, shifted, or special sorted data.

Best For: Rotated arrays, peak elements.

Example: Rotated Sorted Array



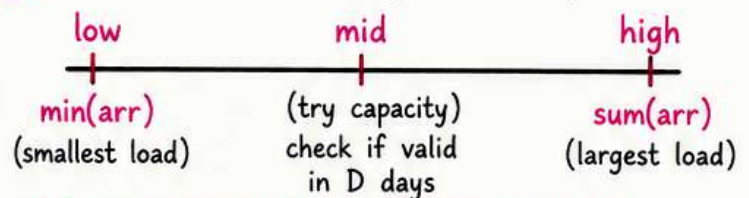
Identify which half is sorted, then apply binary search on that half.

6 Binary Search on Answer

Search over a possible answer range instead of array indices.

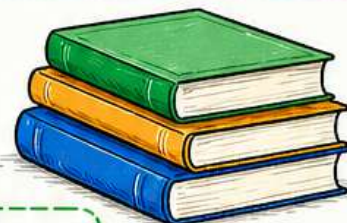
Best For: Minimum capacity, maximum feasible value.

Example: Find minimum capacity to ship all packages in D days.



If valid → try smaller (high = mid - 1)
If not valid → try larger (low = mid + 1)

Note: These searching patterns reduce time complexity and help solve problems efficiently.



Do follow @AdarsChetan on LinkedIn & X for more such amazing stuff!

- @adarschetan

@adarschetan

Do follow me on
LinkedIn & X for
more such
amazing stuff!

INTERVAL & HASHING PATTERNS

7 Merge Intervals

Sort intervals and combine overlapping ranges.

Best For: Meeting schedules, time ranges.

Example:

Input Intervals

[1,3]	[2,6]	[8,10]	[15,18]
-------	-------	--------	---------

After Merging

[1,6]	[8,10]	[15,18]
-------	--------	---------

1-3 and 2-6 overlap → merge into 1-6.
Others remain the same.

8 Hash Map / Frequency Counting

Store values, frequencies, or indices for constant-time lookup.

Best For: Two Sum, anagrams, duplicates.

Example:

Array: [4, 2, 4, 1, 2, 4]

Frequency Map:

Value	1	2	4
Frequency	1	2	3

- Key = value, Value = frequency
- Lookup, insert, update → $O(1)$ average time

9 Prefix Sum

Store cumulative sums to answer range queries efficiently.

Best For: Subarray sums and range queries.

Example:

Array: [3, 1, 4, 1, 5]

Prefix Sum Array:

Index	0	1	2	3	4	5
Prefix Sum	0	3	4	8	9	14

Sum of range [1, 3] (elements 1 to 3)
= $\text{prefix}[4] - \text{prefix}[1] = 9 - 3 = 6$

These patterns help solve many real-world problems by organizing data smartly and answering queries in optimal time.

Do follow @AdarsChetan on LinkedIn & X for more such amazing stuff!

- @adarschetan

@adarschetan
Do follow me on
LinkedIn & X for
more such
amazing stuff! ❤️

LINKED LIST & HEAP PATTERNS



13 Linked List Reversal

Reverse pointers
iteratively or
recursively.

Best For: Reverse list,
palindrome list.

Iterative Approach



Reverse the direction of pointers.



Use three pointers (prev, curr, next)
to reverse the links one by one.

14 In-Place Linked List Manipulation

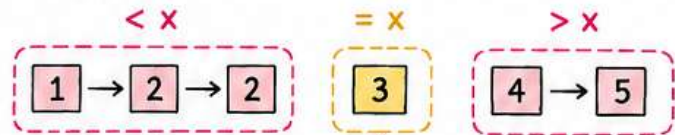
Modify links without using
extra data structures.

Best For: Reordering,
partitioning,
node removal.

Example: Partition List around $x = 3$

Input: 1 → 4 → 3 → 2 → 5 → 2 ($x = 3$)

Process:



Rearrange links so that nodes $< x$ come first,
then $= x$, then $> x$, all in-place.

15 Top K Elements

Use a heap to efficiently
maintain the largest or
smallest K values.

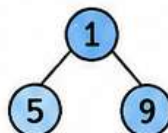
Best For: Kth largest,
frequent elements.

Example: Find Top 3 Largest Elements

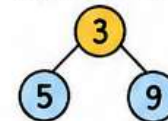
Array: [5, 1, 9, 3, 7, 2, 8] $K = 3$

Use a Min Heap of size K

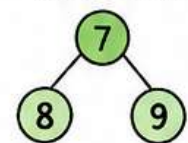
1. Insert first K
elements



2. For each next
element:
If $\text{num} > \text{heap}[1]$,
replace and heapify



3. Final heap
contains Top K
largest elements.



Time Complexity: $O(n \log k)$ | Space Complexity: $O(k)$



Note:

These patterns help you write clean,
optimized code and handle complex
problems with confidence!



Do follow @AdarsChetan on LinkedIn & X
for more such amazing stuff! ❤️



- @adarschetan

@adarschetan

Do follow me on
LinkedIn & X for
more such
amazing stuff! ❤️

STACK & QUEUE PATTERNS



10 Monotonic Stack

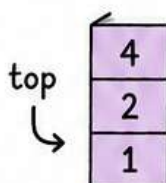
Maintain elements in increasing or decreasing order.

Best For: Next greater element, histogram problems.

Example: Next Greater Element

Array: [2, 1, 2, 4, 3]

2	1	2	4	3
↓	↓	↓	↓	↓
4	2	4	-1	-1



Keep stack in decreasing order.
Pop smaller elements and push current.



11 Stack Simulation

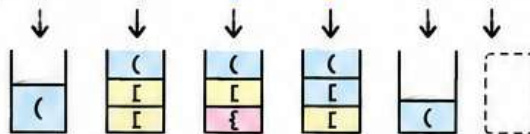
Use a stack to track nested structures or reverse processing order.

Best For: Parentheses, expression evaluation.

Example: Valid Parentheses

String: ([{ }])

Process: ([{ }])



Push opening brackets.
Pop when matching closing bracket found.
If stack empty at end → valid.



12 Monotonic Queue

Maintain useful elements in sorted order inside a moving window.

Best For: Sliding window maximum/minimum.

front →

7	6	3
---	---	---

 ← back
Decreasing Order

Example: Sliding Window Maximum

Array: [1, 3, -1, -3, 5, 3, 6, 7] k = 3

1	3	-1	-3	5	3	6	7
↓	↓	↓	↓	↓	↓	↓	↓
3	3	5	5	6	7		

Window max

Remove out-of-window elements and keep queue in decreasing order.



Note:

These patterns make your code faster, cleaner and help solve complex problems with ease!



Do follow @AdarsChetan on LinkedIn & X for more such amazing stuff!



- @adarschetan

@adarschetan
Do follow me on
LinkedIn & X for
more such
amazing stuff! ❤️

RECURSION & DYNAMIC PROGRAMMING



20 Backtracking

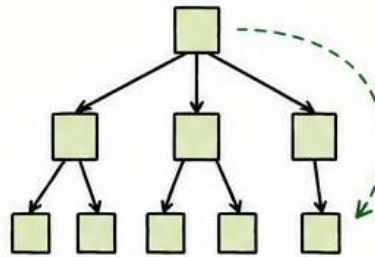
Build solutions incrementally and undo choices when necessary.

Best For: Permutations, combinations, Sudoku.



How it works:

1. Choose
2. Explore
3. Unchoose (Backtrack)



Example: N-Queens (N = 4) ★

Q			
		Q	
	Q		
			Q

Try placing queens row by row. If a placement is invalid, backtrack and try another. ↻

21 1D Dynamic Programming

Store previous results to avoid repeated calculations.

Best For: Climbing stairs, House Robber.



Example 1: Climbing Stairs

You can climb 1 or 2 steps.

Ways to reach step n:

$$dp[n] = dp[n-1] + dp[n-2]$$

DP Table:

n	0	1	2	3	4	5
dp[n]	1	1	2	3	5	8

Time: $O(n)$ | Space: $O(n)$ → can be optimized to $O(1)$



22 2D Dynamic Programming

Use a table to represent solutions involving two changing states.

Best For: LCS, edit distance, grid problems.



Example: Longest Common Subsequence (LCS)

Text1: A B C D A B

Text2: B D C A B A

i \ j	B	D	C	A	B	A
0	0	0	0	0	0	0
A	0	1	0	0	1	1
B	0	0	1	1	1	2
C	0	1	1	2	2	2
D	0	1	2	2	2	2
A	0	1	2	2	3	3
B	0	1	2	3	3	4

$dp[i][j]$ = length of LCS of Text1[0..i-1] and Text2[0..j-1]

Answer (LCS length) = $dp[n][m] = 4$ (LCS: BDAB)

Time: $O(n*m)$ | Space: $O(n*m)$



Note: ❤️

These patterns are the building blocks of problem solving. Practice them regularly and master one by one!



Do follow @AdarsChetan on LinkedIn & X for more such amazing stuff! ❤️



- @adarschetan

@adarschetan
Do follow me on
LinkedIn & X for
more such
amazing stuff! ❤️

ADVANCED INTERVIEW PATTERNS



23 Greedy

Choose the best local decision at each step.

Best For: Scheduling, intervals, optimization problems.



Example: Activity Selection

Select maximum number of non-overlapping activities.

Activities (start, end):

[1,3] [2,5] [4,6] [6,7] [5,8]

Greedy Choice: Pick activity with earliest finish time.

Order of Selection → [1,3] → [4,6] → [6,7]

Idea:

Always pick the option that leaves maximum room for future choices.



24 Union Find (DSU)

Track connected components using parent and rank/size arrays.

Best For: Connectivity, redundant edges, Kruskal's algorithm.

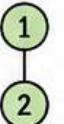


How it works:

Each node is its own parent initially.

Use union by rank and path compression for efficiency.

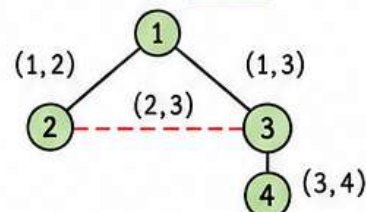
	1	2	3	4	5
parent	1	2	3	4	5
rank	0	0	0	0	0

Union(1, 2) →  Find(2)
Returns root of 2 → 1

Example: Redundant Connection

Edges added one by one:

(1, 2), (1, 3), (2, 3) (3, 4)



When adding (2, 3), 2 and 3 are already connected → cycle detected. So (2, 3) is redundant.



25 Trie

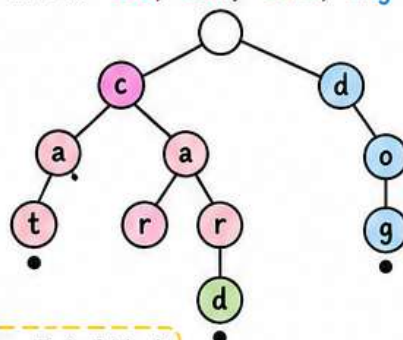
Store strings character by character for efficient prefix searching.

Best For: Autocomplete, dictionary search, word problems.



Example: Insert words

Words: cat, car, card, dog



• = End of Word

Operations:

- ✓ **Insert:**
Add each character, create node if not exists.
- 🔍 **Search:**
Follow path, return true if end node found.
- ★ **Starts With:**
Check if prefix path exists (end node not required).

Note: ★

Master these patterns to solve complex problems faster and crack any technical interview with confidence! ❤️



Do follow @AdarsChetan on LinkedIn & X for more such amazing stuff!



- @adarschetan